

Your Iceberg Table Is Already a Pinot Table

A Netflix format, a LinkedIn engine, and
why they fit

Gonzalo Ortiz · Apache Pinot PMC · StarTree



Quick show of hands



Written a Parquet file?



Know what is in its footer?



Used Iceberg in production?



Heard of Apache Pinot?

CHAPTER 0

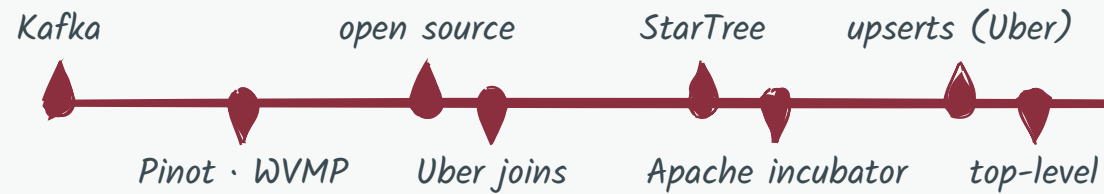
Two lines

Two projects, two companies, one design

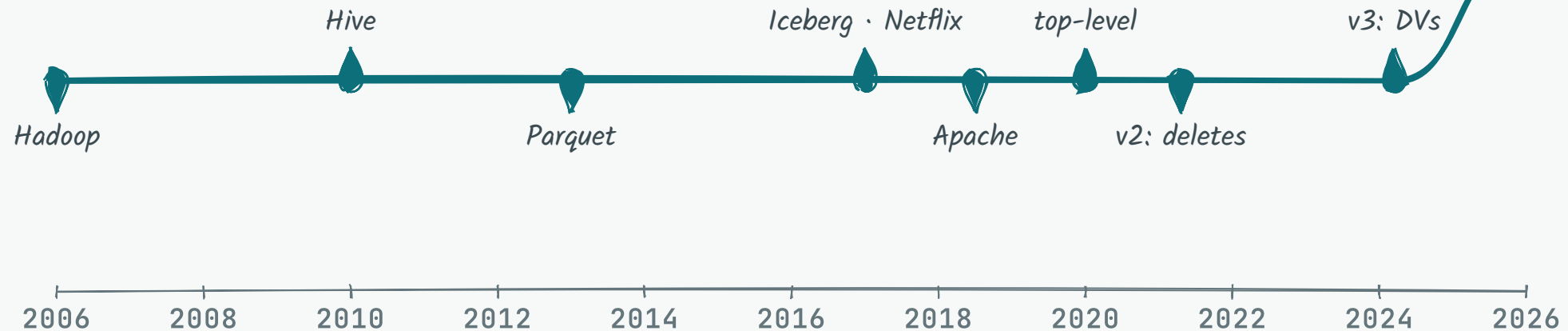


Two timelines that meet

LINKEDIN → APACHE PINOT



THE DATA LAKE → APACHE ICEBERG



External Tables

2025

CHAPTER 1

Iceberg

A table format, built bottom-up



youtube.com

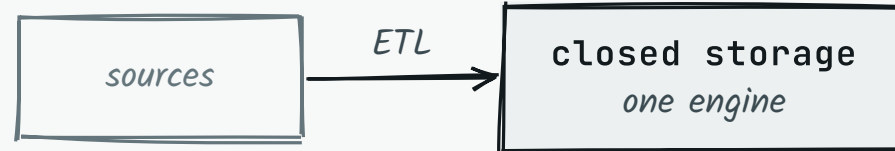
RECOMMENDED WATCH

Apache Iceberg: What It Is and Why Everyone's Talking About It

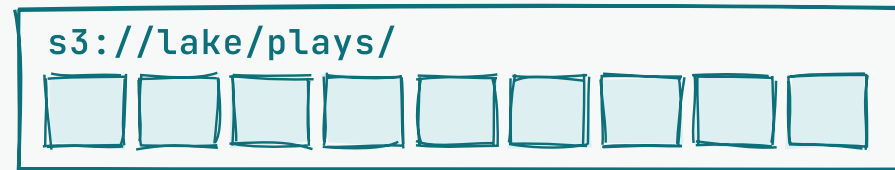
Confluent Developer · YouTube

Warehouse, lake, lakehouse

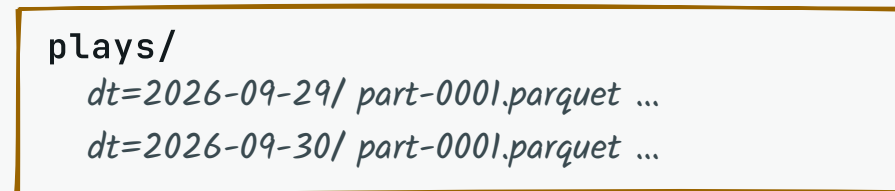
WAREHOUSE



LAKE

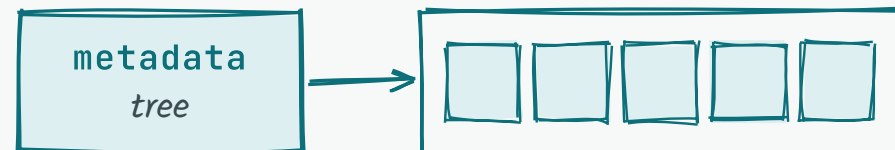


HIVE



*a table is
a directory*

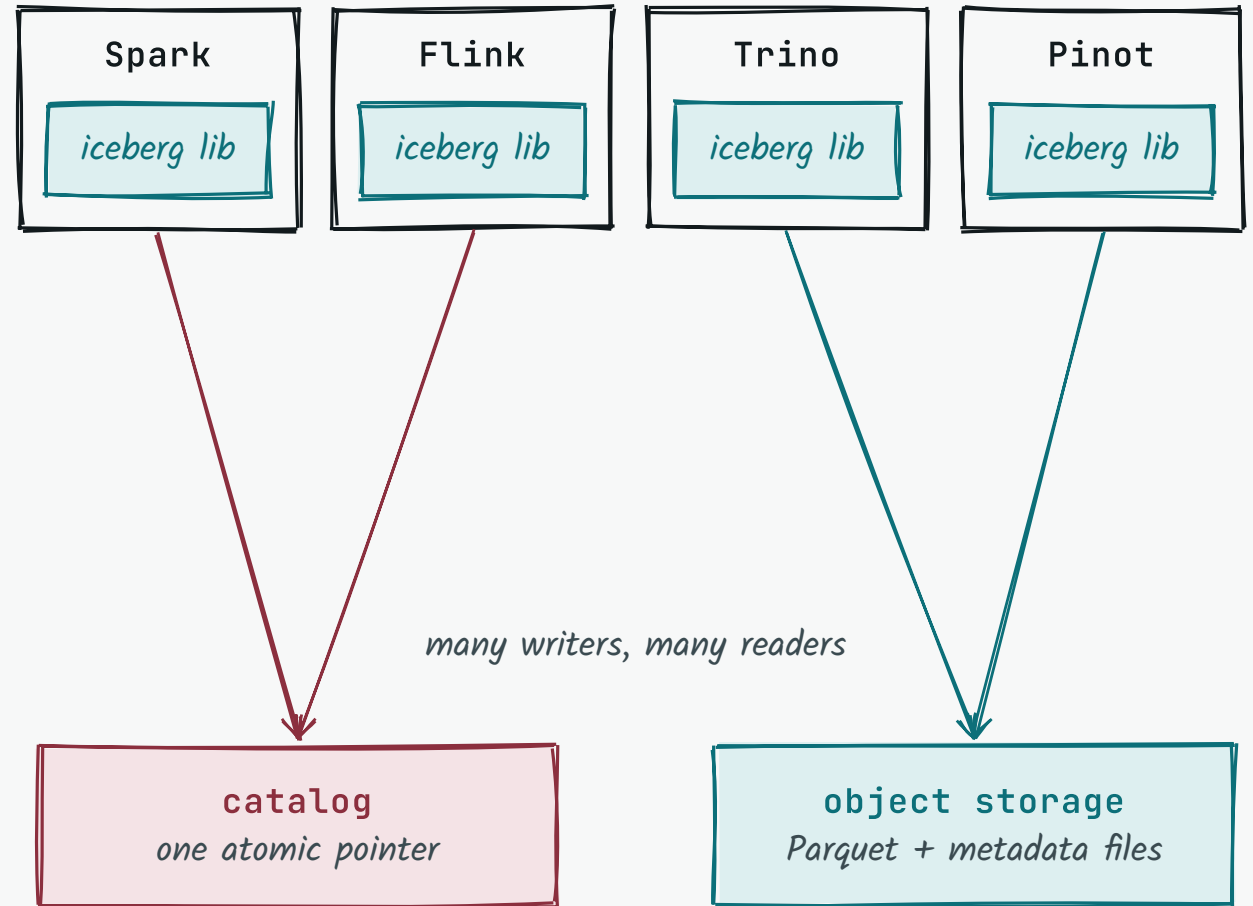
LAKEHOUSE



real tables

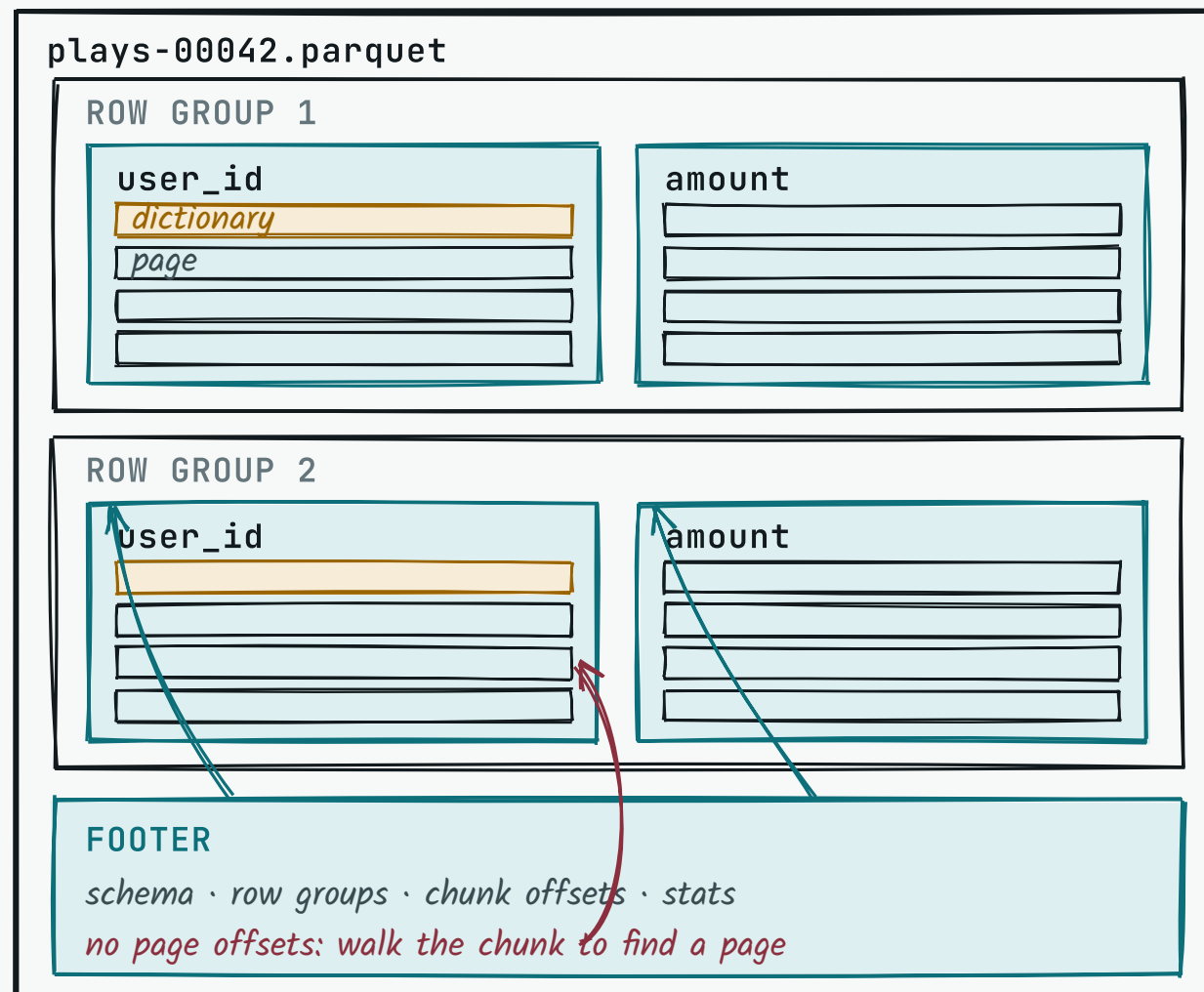
A format, not an engine

- No Iceberg server
- No coordinator
- A spec plus libraries
- Every engine embeds them
- Many writers, many readers



Parquet 101

- **Row group:** horizontal slice
- **Column chunk:** one column, contiguous
- **Page:** smallest readable unit
- **Footer:** offsets, stats, schema
- No page offsets in the footer



Building a table, bottom-up

CATALOG

db.plays → v3

the only mutable thing

METADATA

v3.metadata.json
schema · spec · snapshots

schemas · snapshots

MANIFEST LIST

snap-3.avro
day: 09-28 ... 09-30

partition ranges

MANIFESTS

manifest A
user_id 3 ... 9,814

manifest B
user_id 12 ... 9,977

per-file column min/max

DATA FILES

parquet

parquet

parquet

parquet

parquet

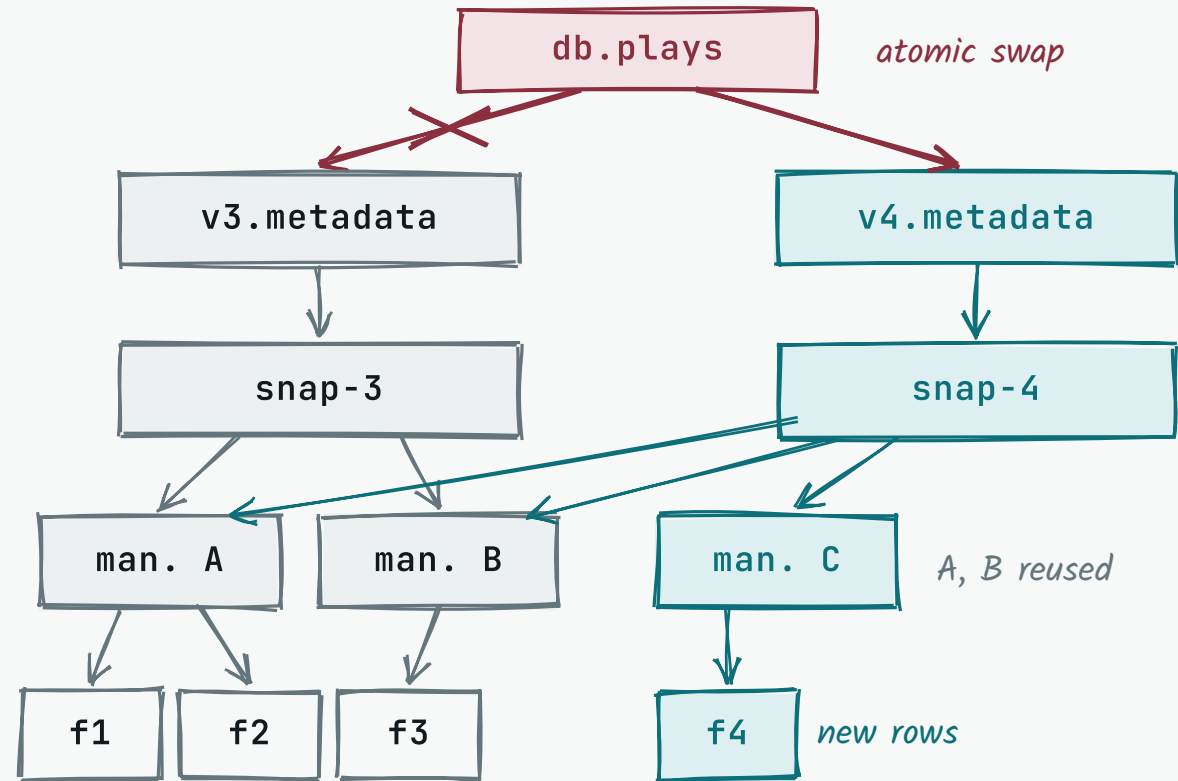
immutable Parquet

ICEBERG

Iceberg is a B-tree of files where only the root pointer is mutable.

A commit

1. Write new data files
2. Write new manifests
3. Write new metadata.json
4. Swap the catalog pointer
5. Lost the race? Retry

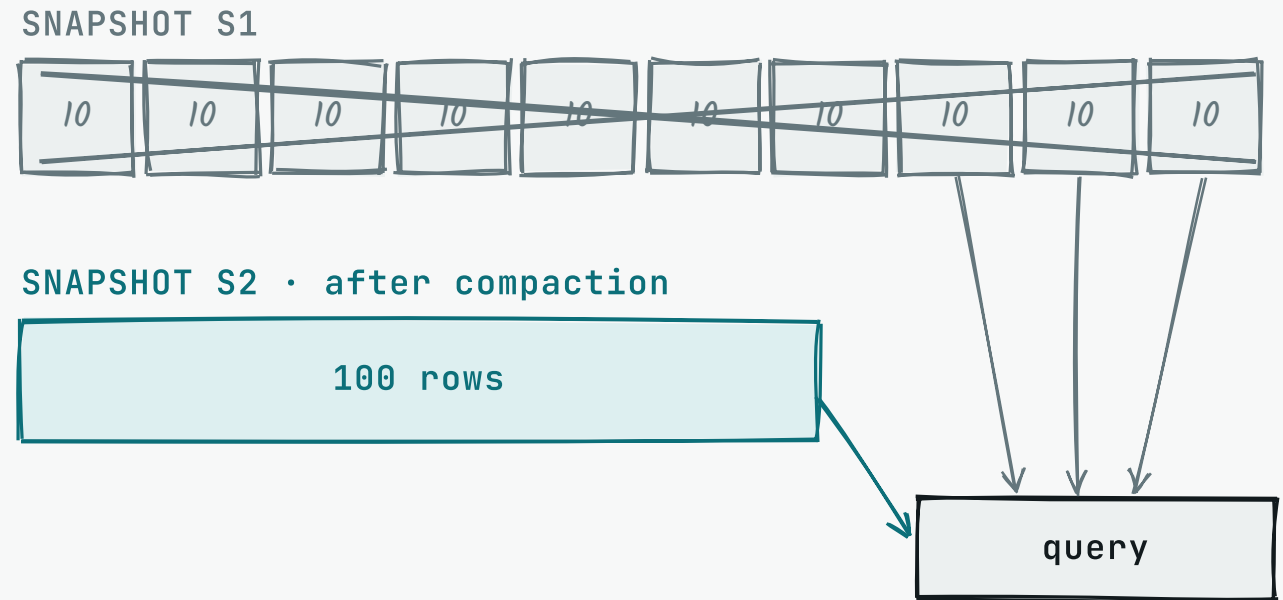


Someone swapped first? The swap fails.

Re-read the new tree, re-apply the change **if possible**, try again.

Never mix two snapshots

- Compaction: 10 files become 1
- Mix before and after?
- Rows twice, or not at all
- Every read pins one snapshot

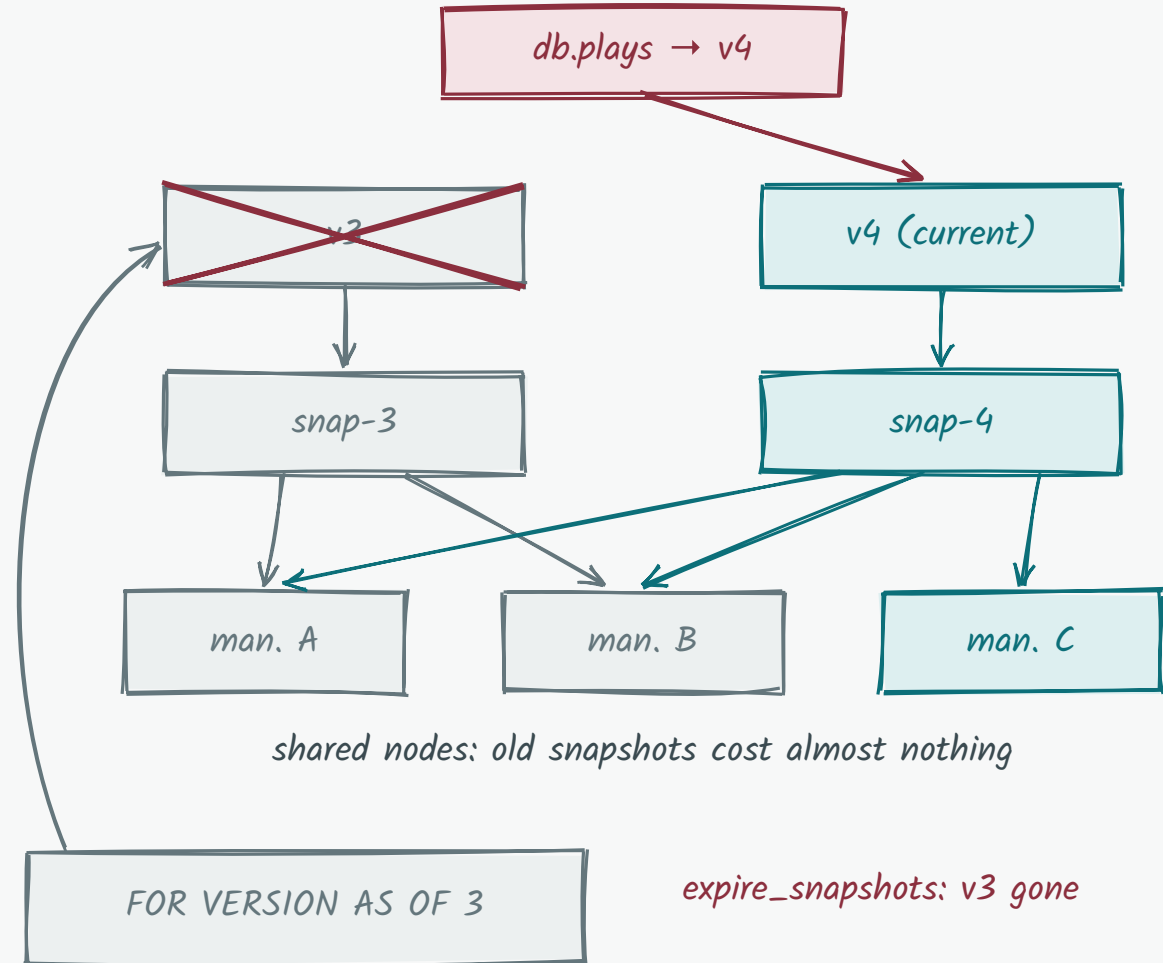


count = 140
S1 + S2 mixed: wrong

pin S2 → count = 100

Time travel

- Old trees stay reachable
- FOR VERSION AS OF
- Branches and tags
- Until snapshots expire



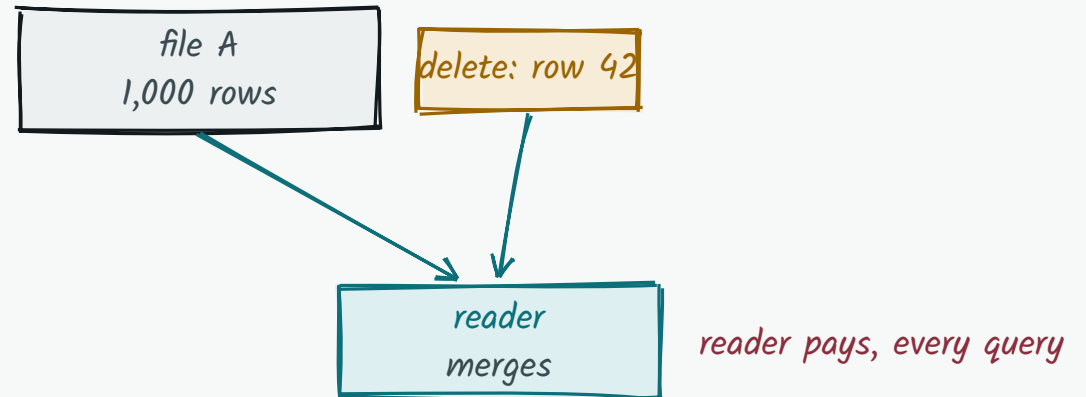
Deletes: who pays?

- **Copy-on-write:** writer rewrites files
- **Merge-on-read:** writer adds a delete file
- Reader merges at scan time
- Compaction folds them back

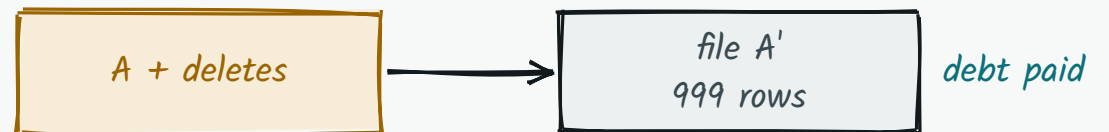
COPY-ON-WRITE



MERGE-ON-READ



COMPACTION



Merge-on-read, three ways

- Position deletes **v2**: file + row
- Equality deletes **v2**: "where id = 42"
- Deletion vectors **v3**: one bitmap per file

POSITION DELETES · v2

file pos

s3://.../data-0007.parquet 42

s3://.../data-0007.parquet 913

which row of which file

EQUALITY DELETES · v2

user_id = 4217

track_id = 99

every row matching

→ join on every read

DELETION VECTORS · v3

PUFFIN FILE



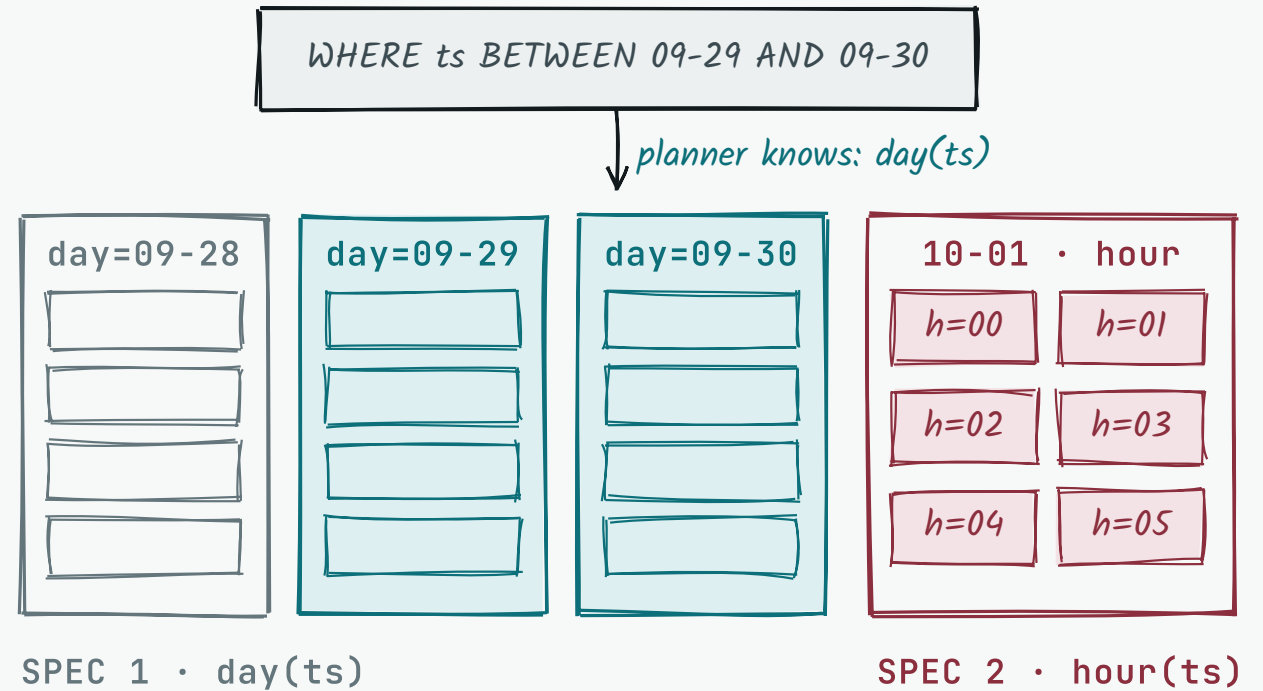
one roaring bitmap per data file

*one bitmap AND
at most one per file*

Puffin: a blob file format from the Iceberg spec. It stores DVs and statistics.

Hidden partitioning

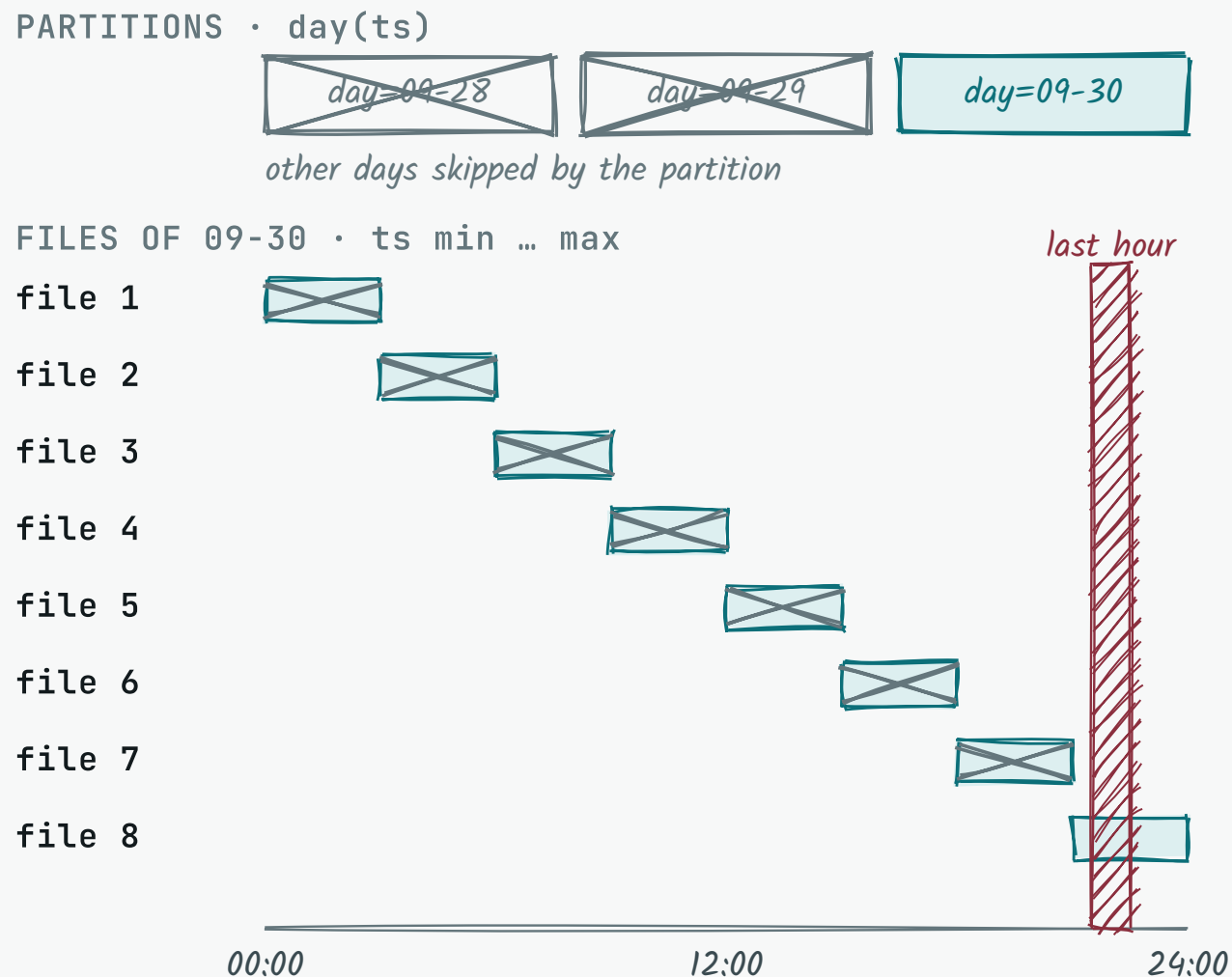
- Partition by a transform
- `day(ts)`, `bucket(16, user_id)`
- Users filter on `ts`
- Change the spec, keep old files



old files keep spec 1, nothing is rewritten

Why min/max helps here

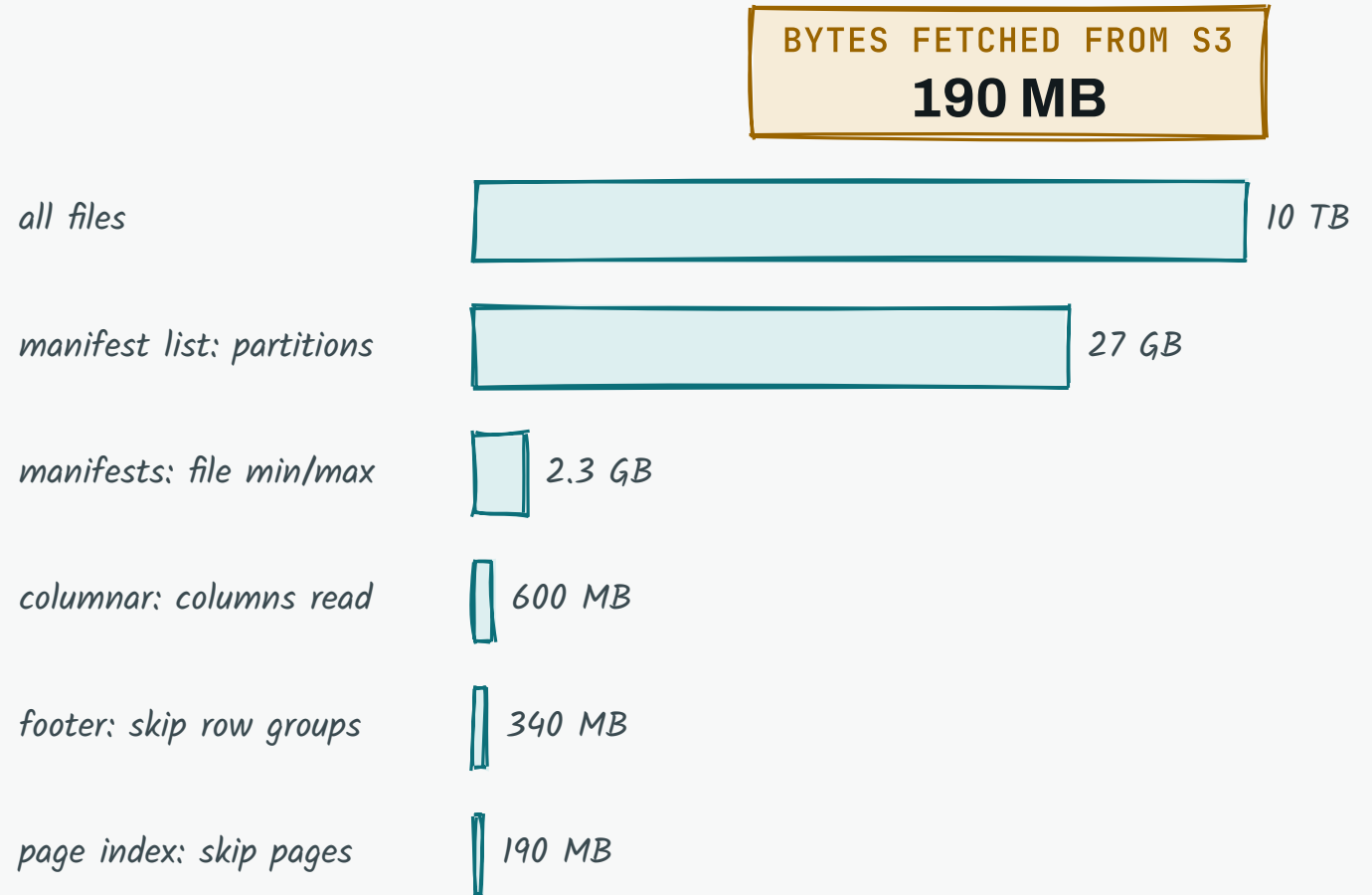
- Partition: one day, not the year
- Inside the day: files in time order
- Query: the last hour
- One file overlaps: skip the rest



When pruning works

- Manifest list: partitions
- Manifests: file min/max
- Columnar: needed columns only
- Footer: skip row groups
- Page index: skip pages

```
WHERE ts >= now() - INTERVAL 1 HOUR
```



Where min/max gives up

- Partition: 7 days, 1 bucket of 16
- Inside the bucket: `user_id` spread out
- Every range contains it
- Min/max skips nothing

PARTITIONS · 7 days × bucket(16, user_id)



1 bucket of 16, 7 days of 365: much less data

FILES OF BUCKET 7

file 1

file 2

file 3

file 4

file 5

file 6

file 7

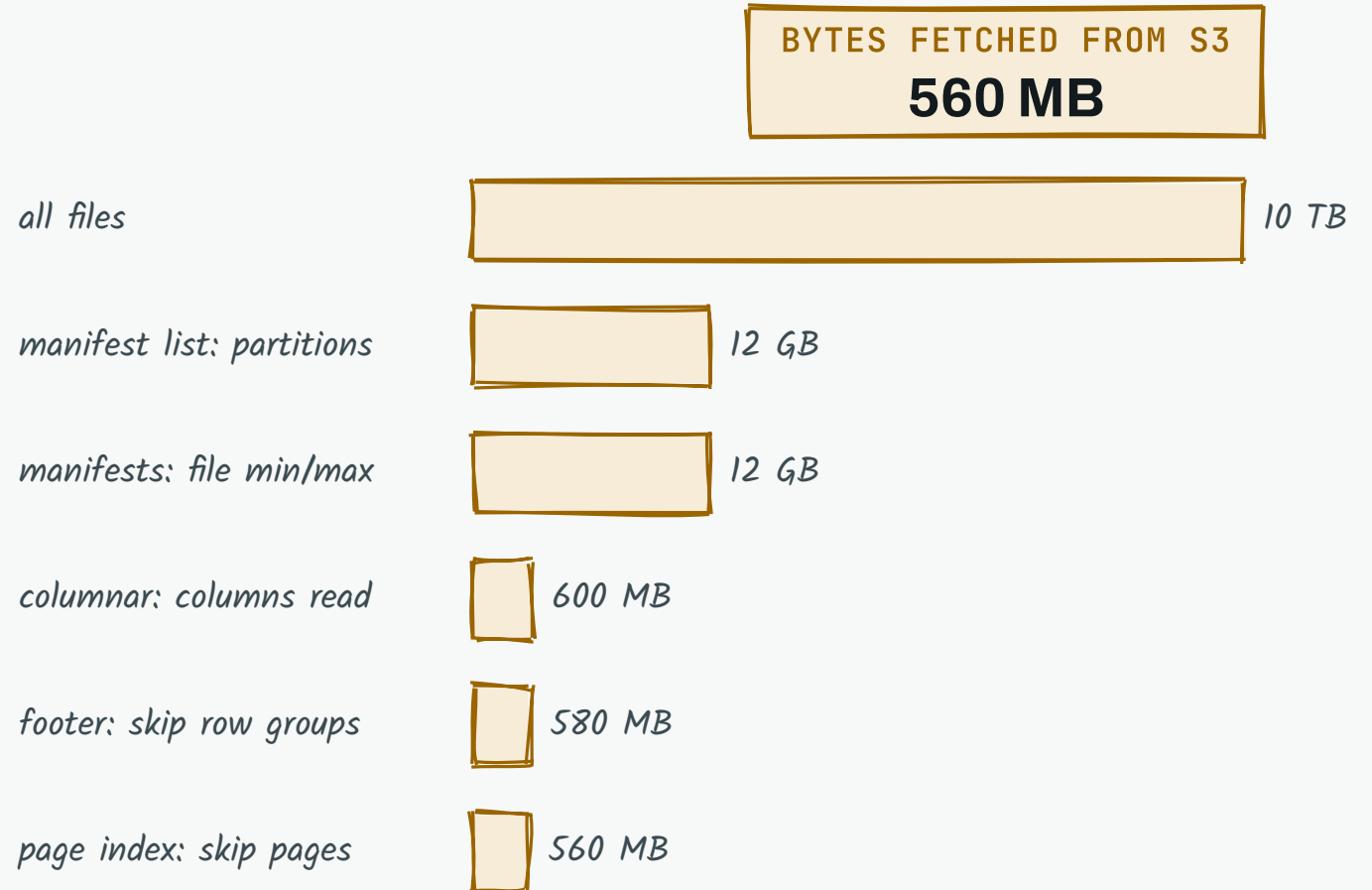
file 8



When pruning does not

- Filter on `ts` and `user_id`
- Partitions: a big cut
- File min/max: no help
- Columnar still helps
- ~560 MB for a few rows

```
WHERE ts >= now() - 7 DAY AND user_id = 4217
```



How engines read Iceberg

ENGINE	LOCAL COPY	REMOTE SCAN + CACHE
Trino / Athena / Spark	—	✓
DuckDB	✓	✓
Dremio	✓ Reflections	✓ C3 cache
Snowflake / BigQuery	✓	✓
StarRocks / Doris / ClickHouse	✓	✓ data cache
Databricks	✓	✓ disk cache

Official documentation of each engine, checked 2026-10-01.

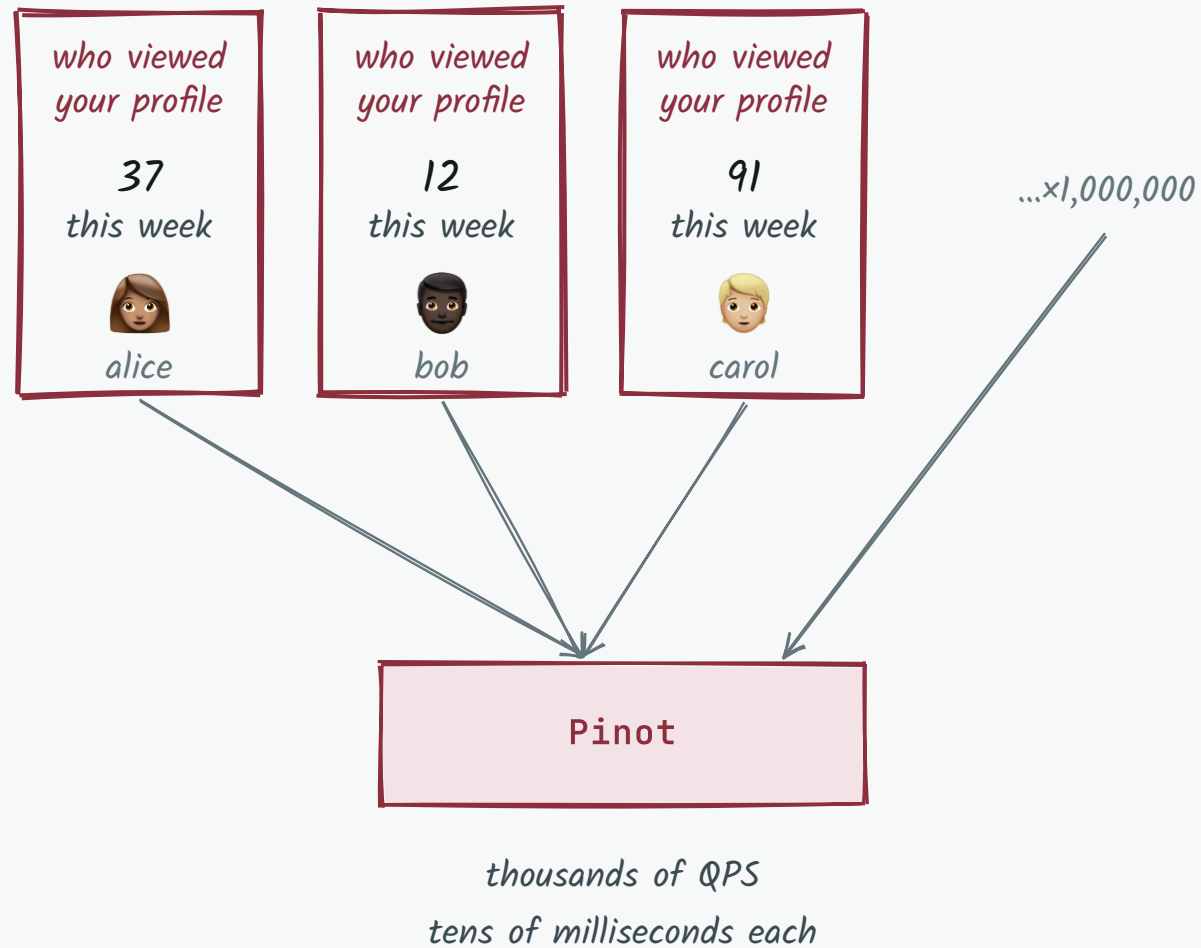
CHAPTER 2

Pinot

Analytics as a product feature

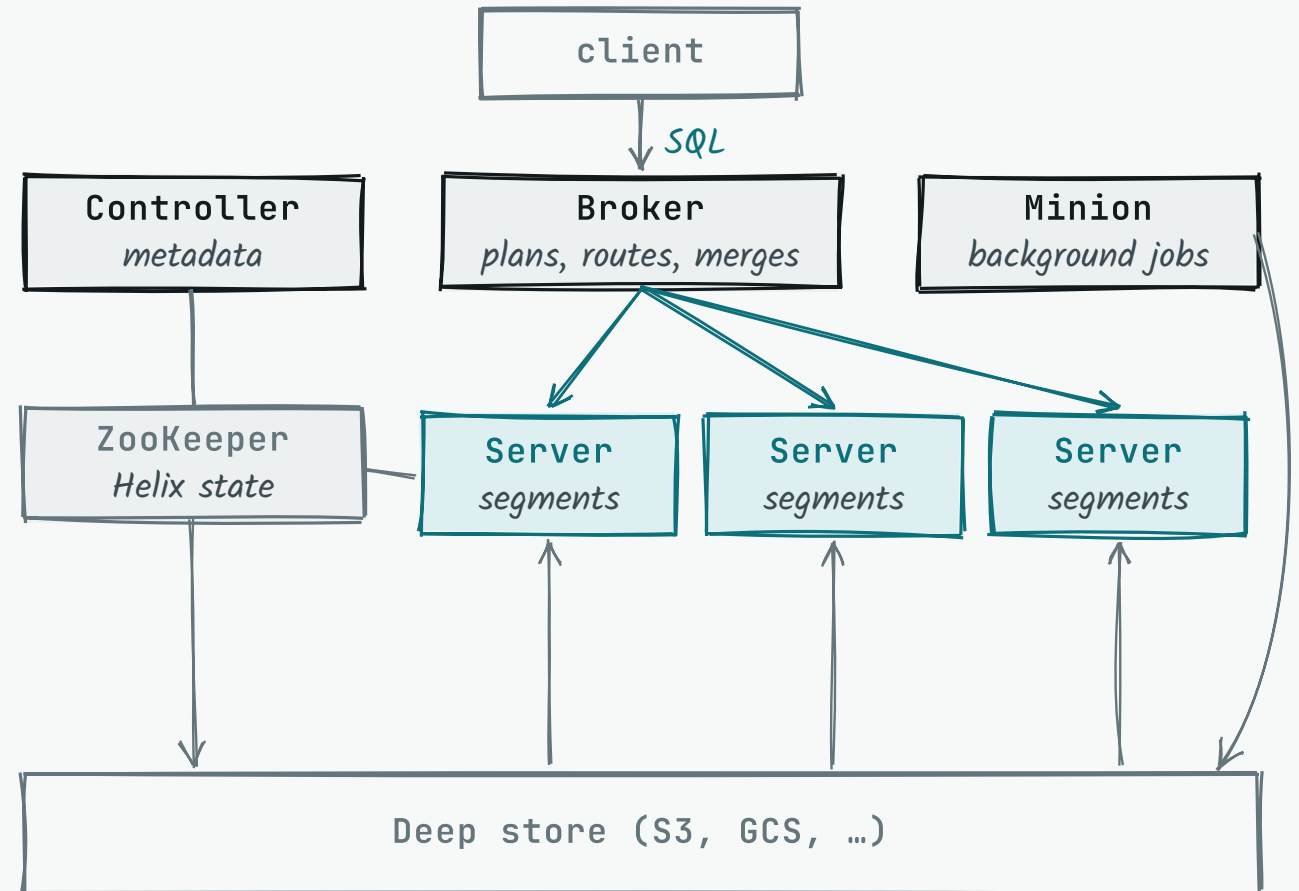


Why Pinot exists



The moving parts

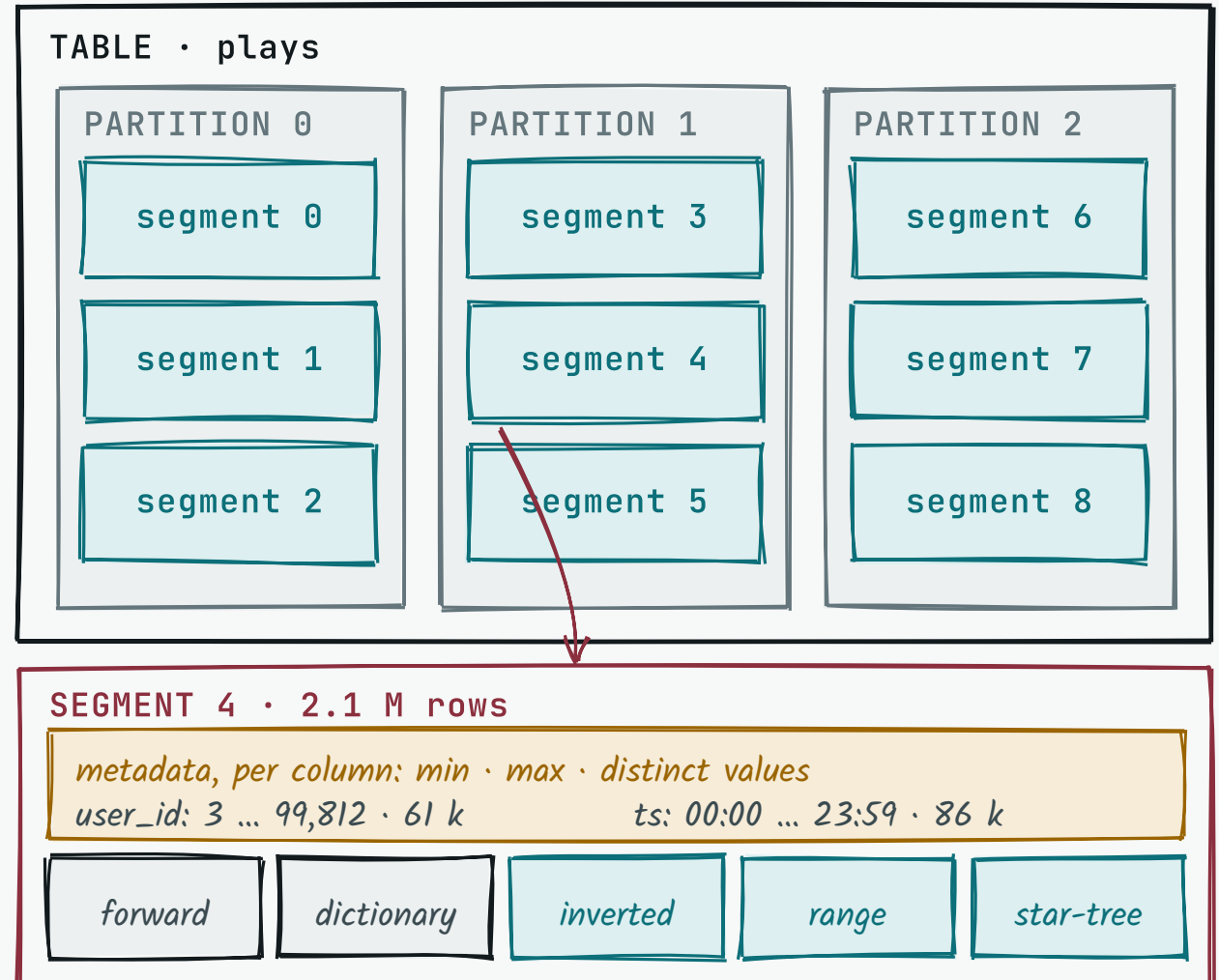
- **Controller:** metadata, assignment
- **Broker:** plans, routes, merges
- **Server:** stores, executes
- **Minion:** background jobs
- ZooKeeper + deep store



segments: immutable files in the deep store, loaded by servers

Tables all the way down

- Table
- Partitions
- Segments: immutable, columnar
- Metadata + indexes per column



Min/max versus an index

- Min/max: "which files might"
- Inverted index: "which rows"
- Star-tree: pre-aggregated answers

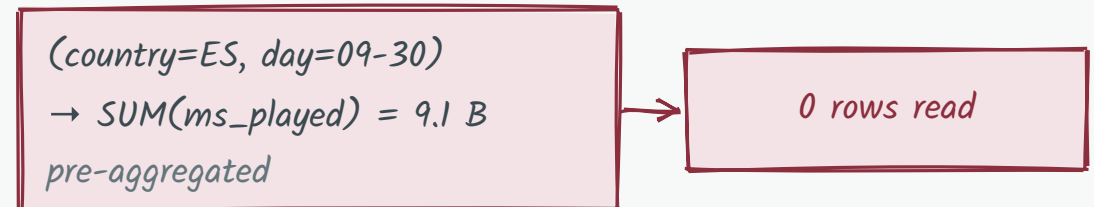
MIN/MAX: WHICH FILES MIGHT



INVERTED INDEX: WHICH ROWS

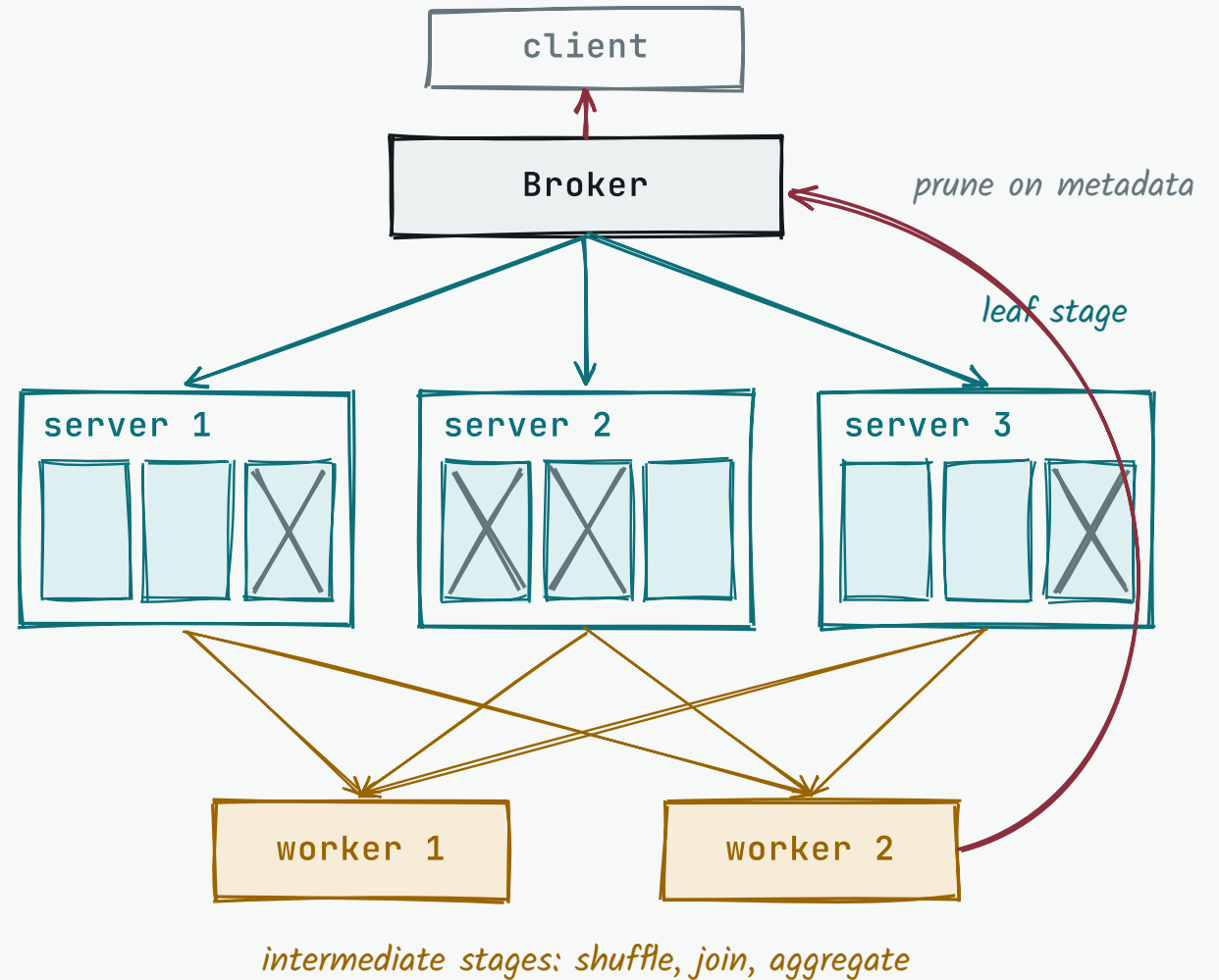


STAR-TREE: THE ANSWER ITSELF



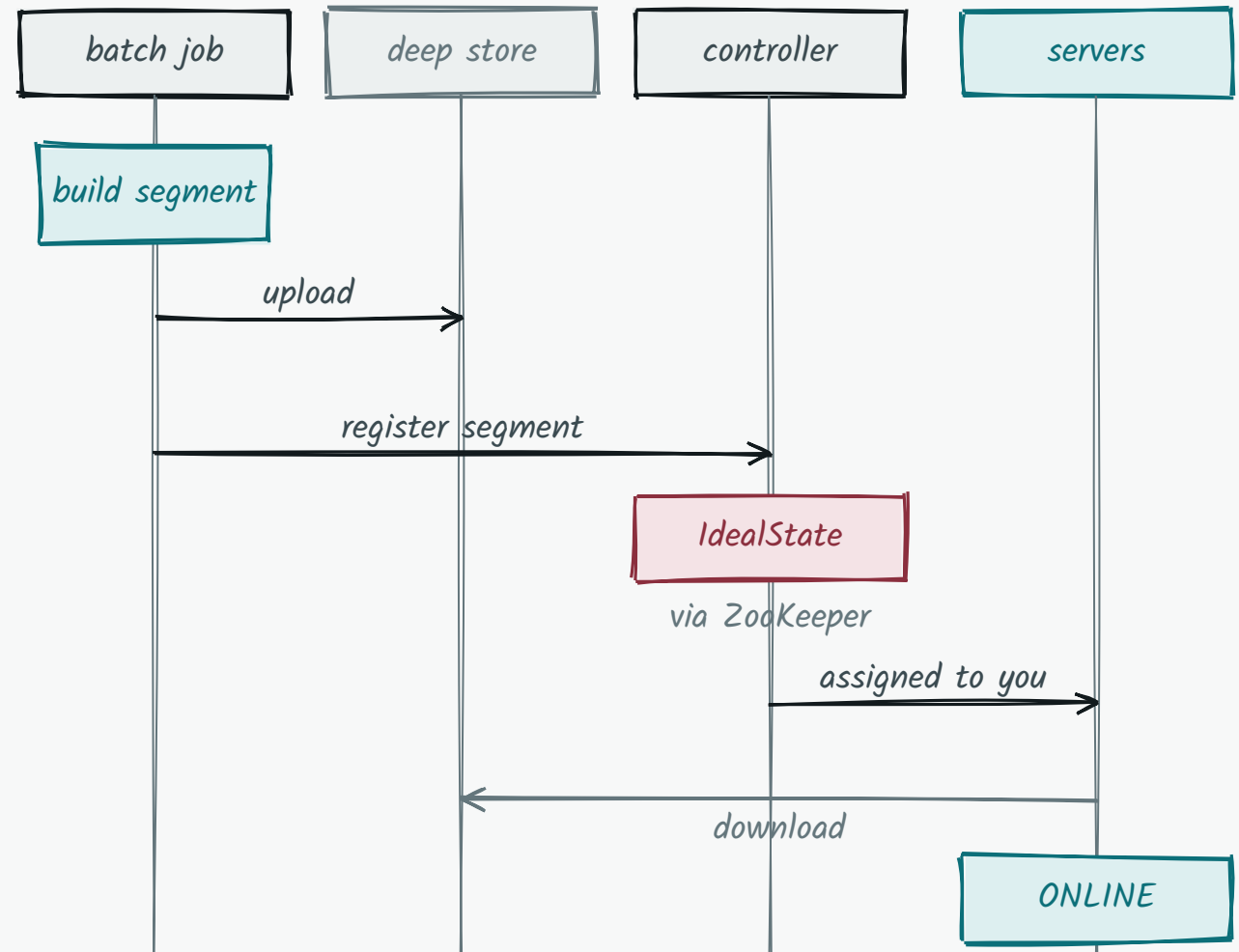
A SQL query, end to end

1. Broker prunes segments
2. Leaf stage: servers scan with indexes
3. Intermediate stages: shuffle, join, aggregate
4. Broker returns the result



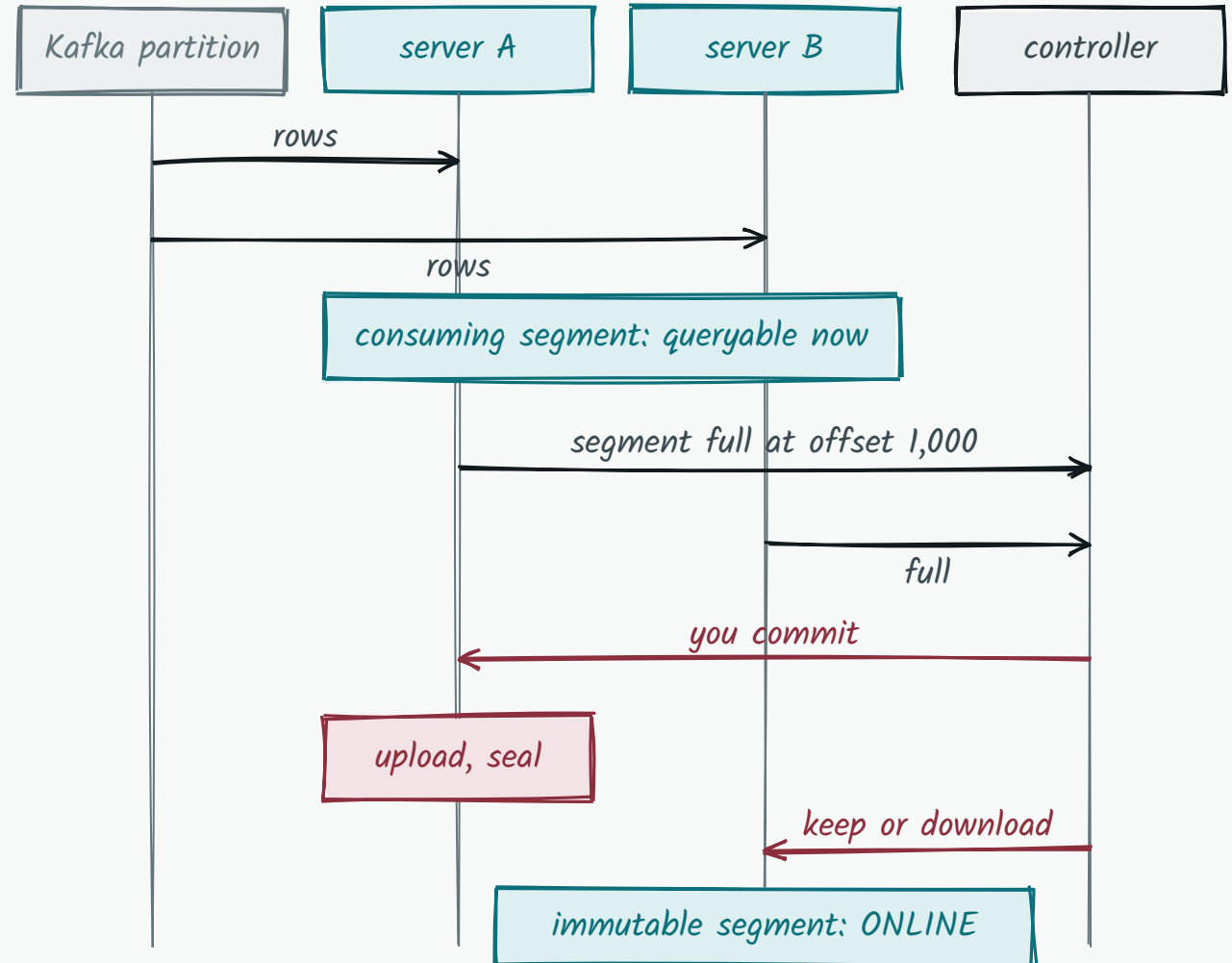
Offline ingestion

1. Job builds a segment
2. Upload to deep store
3. Controller registers it
4. Servers download and serve



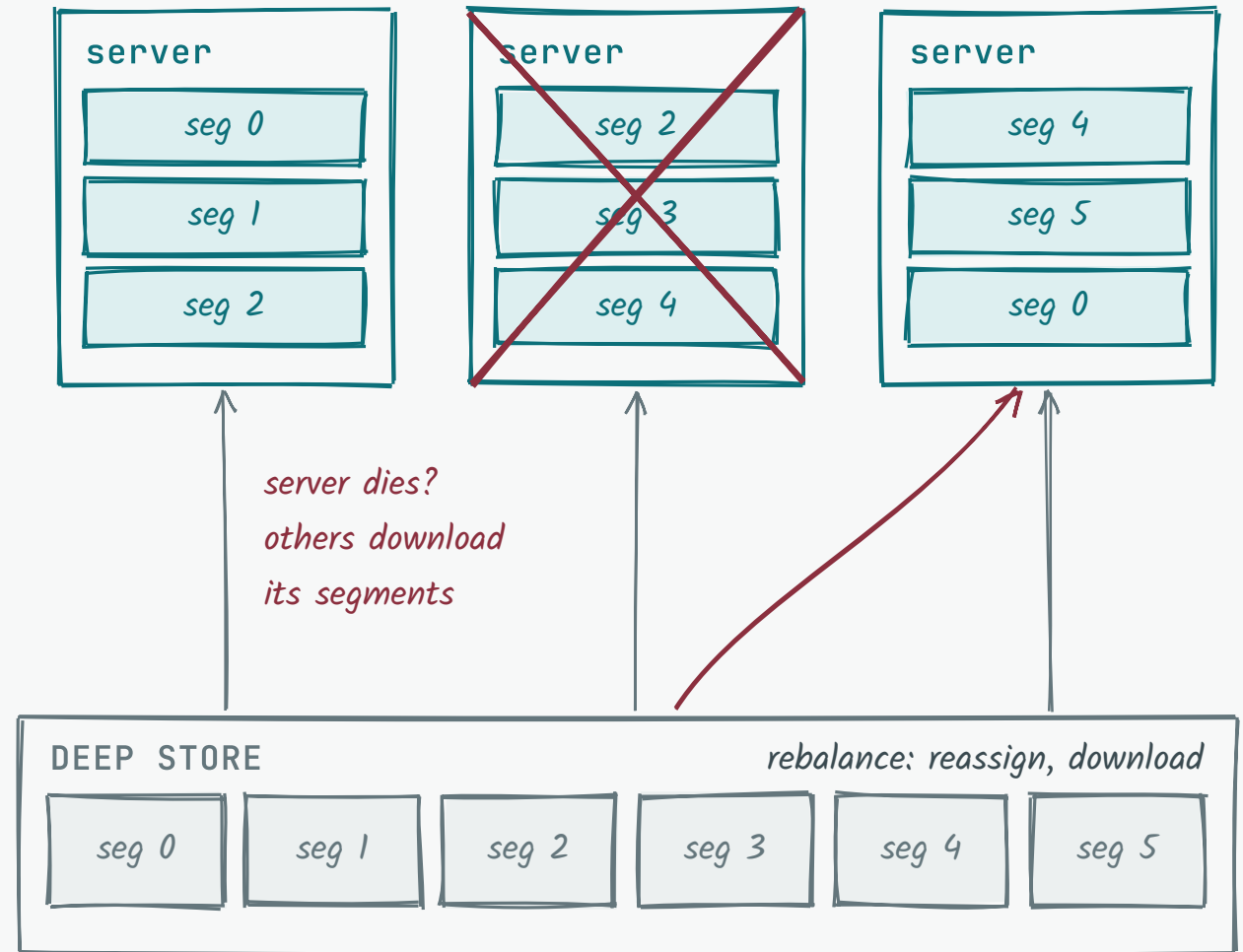
Realtime ingestion

1. Servers consume a Kafka partition
2. Rows are queryable at once
3. Segment fills up
4. One replica commits it
5. It becomes immutable



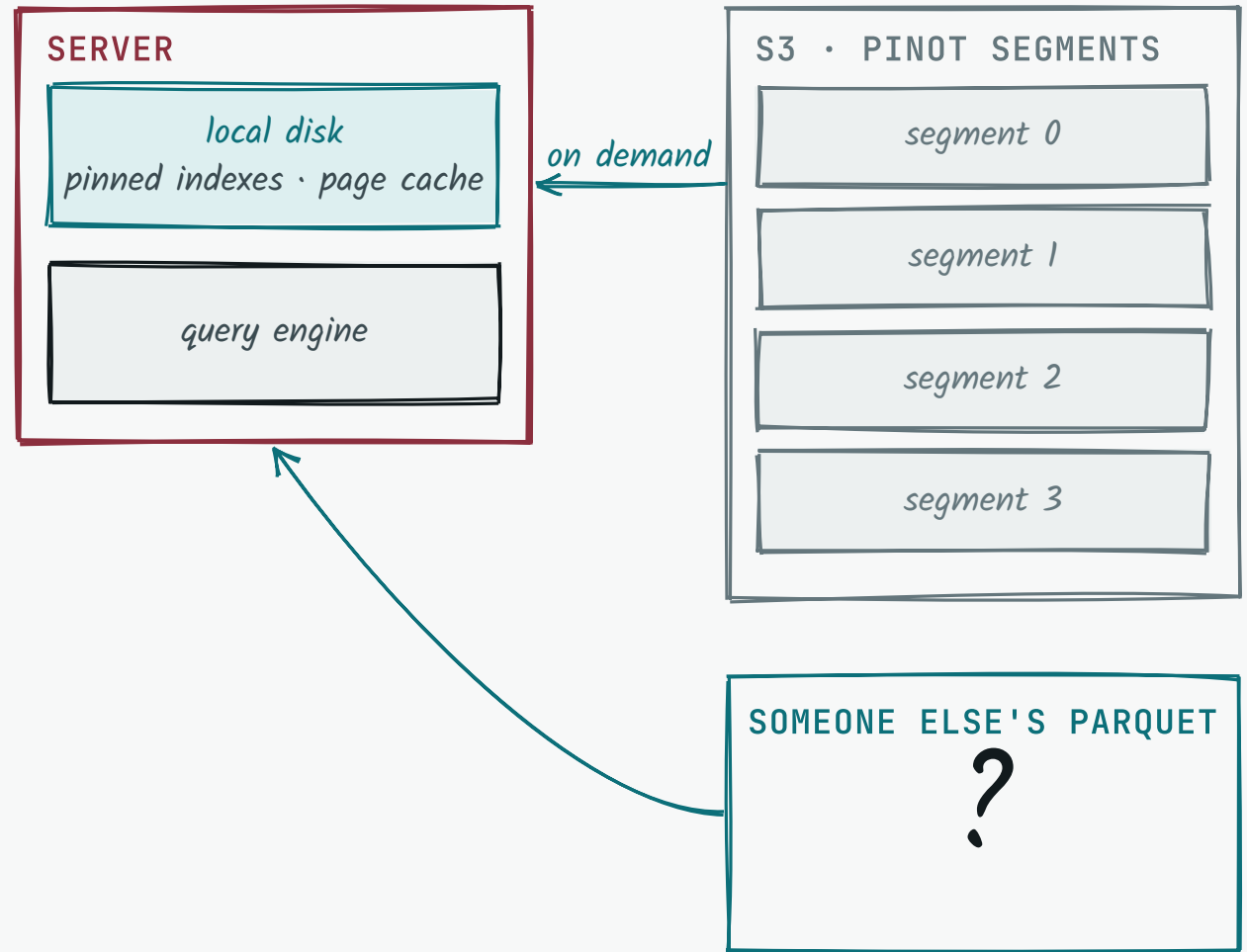
Segments live in the deep store

- Every segment is in the deep store
- Servers keep a local copy
- Server dies? Download again
- Rebalance: move assignments



Tiered storage

- Skip the local copy
- Read from S3 on demand
- Less disk, more latency
- Pin hot indexes locally
- What if the files were not ours?



CHAPTER 3

Iceberg ♥ Pinot

Same shape, so no copy

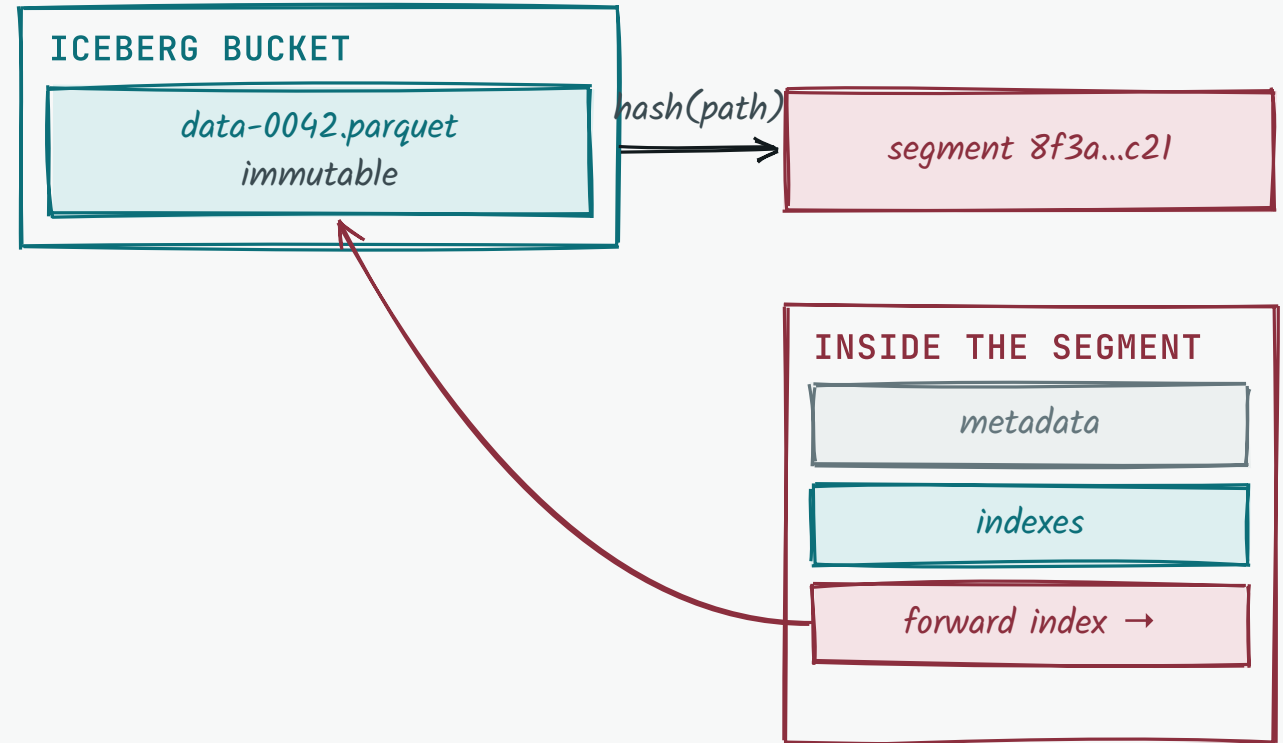


Same shape

ICEBERG	PINOT
Data file, immutable	Segment, immutable
Manifest stats	Segment metadata
Snapshot	A routable segment set
Partition spec	Segment partitioning
Deletion vector	Valid-doc bitmap (upserts)
Snapshot expiry	Retention

One Parquet file, one Pinot segment

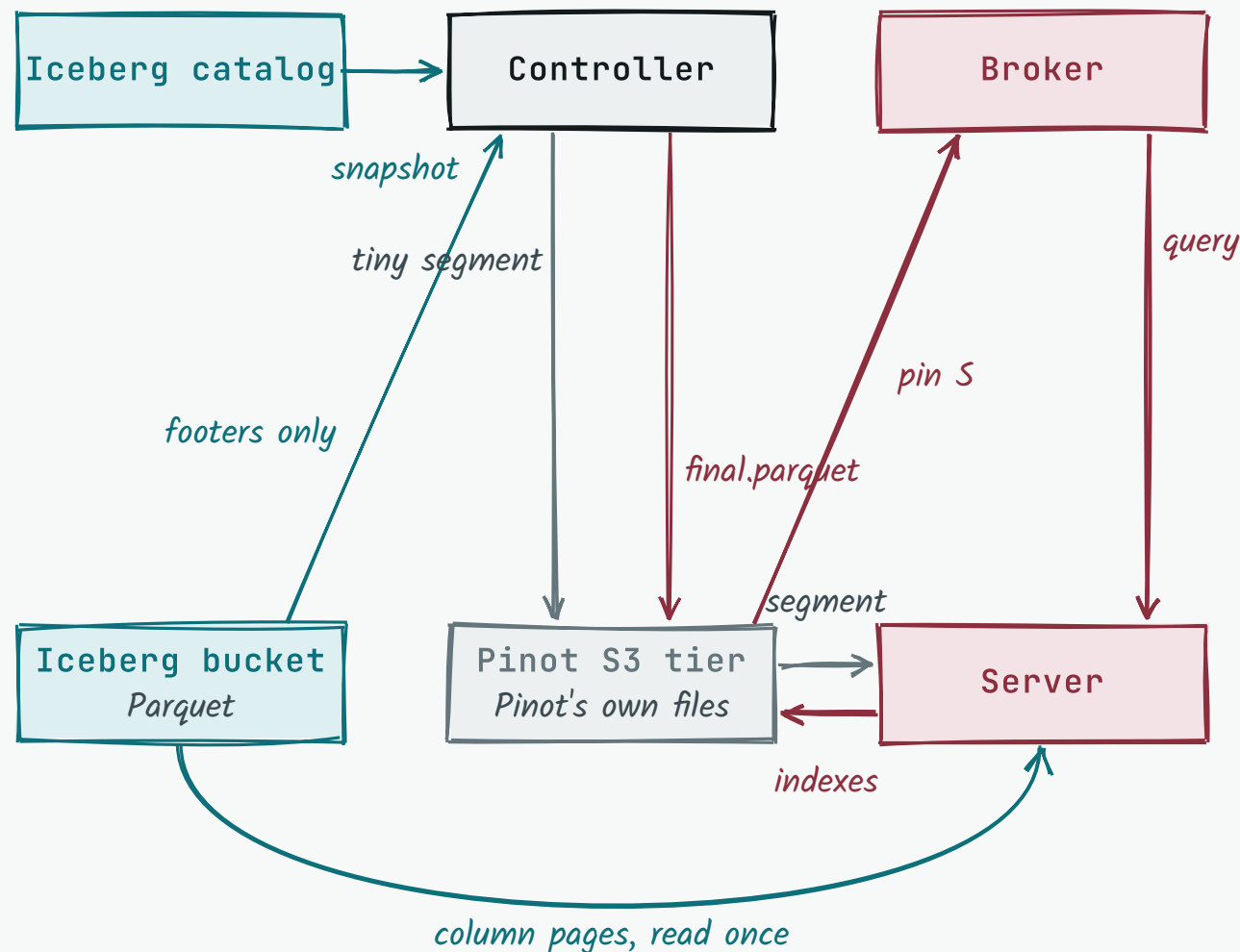
- Name = hash of the file path
- Paths never change content
- Segment holds no data
- Its forward index is the Parquet file



*no data inside
the forward index is the file*

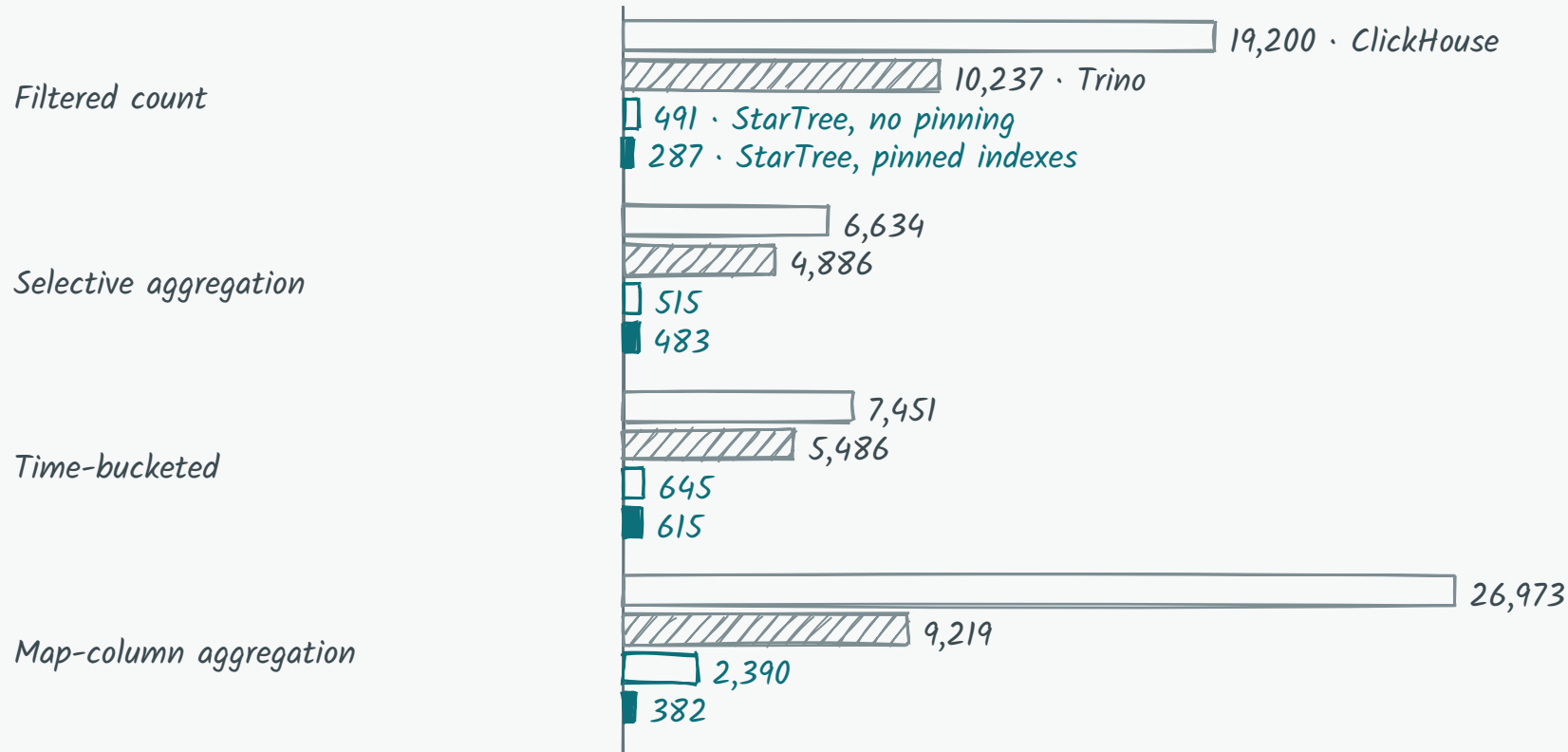
From catalog to query

1. Controller reads the snapshot
2. New file: footer only, tiny segment
3. Server builds indexes, stores them on S3
4. Snapshot published
5. Broker pins it per query



Same Parquet, same hardware

LATENCY, MS • NO DATA CACHE • LOWER IS BETTER



NETWORK IN

4.2 GB

StarTree

~20 GB

Trino • ClickHouse

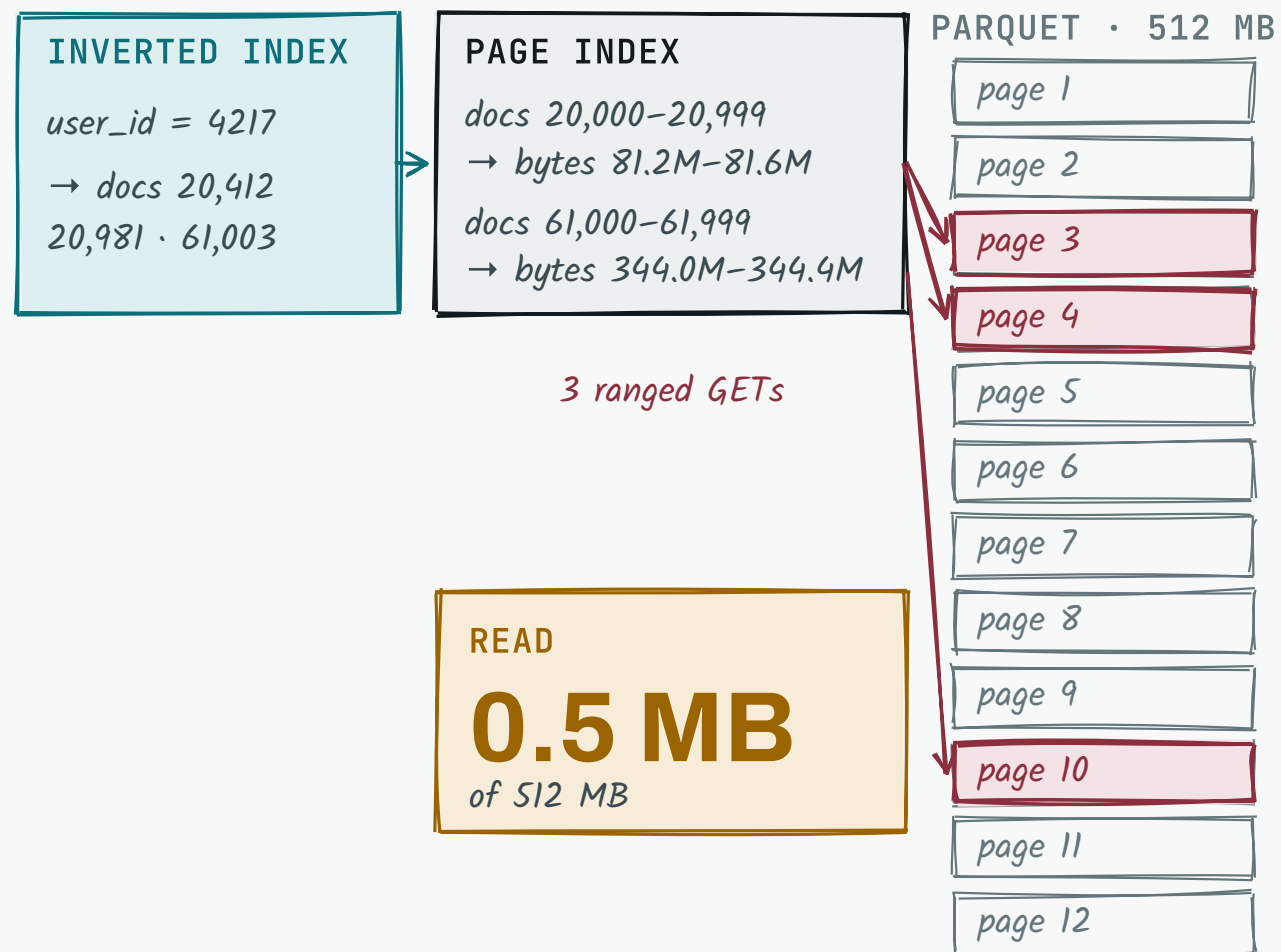


StarTree benchmark: 12.2 B rows, 770 GB Parquet on S3, 4 × r6gd.4xlarge for every engine. Trino 476 (Iceberg connector), ClickHouse 26.2 (icebergS3Cluster). Indexes add 270 GB.

startree.ai • vs Trino, ClickHouse

Filtering with an index

- Indexes give exact doc IDs
- Page index gives byte ranges
- Ranged GETs, only those pages
- 512 MB file, 0.5 MB read



startree.ai

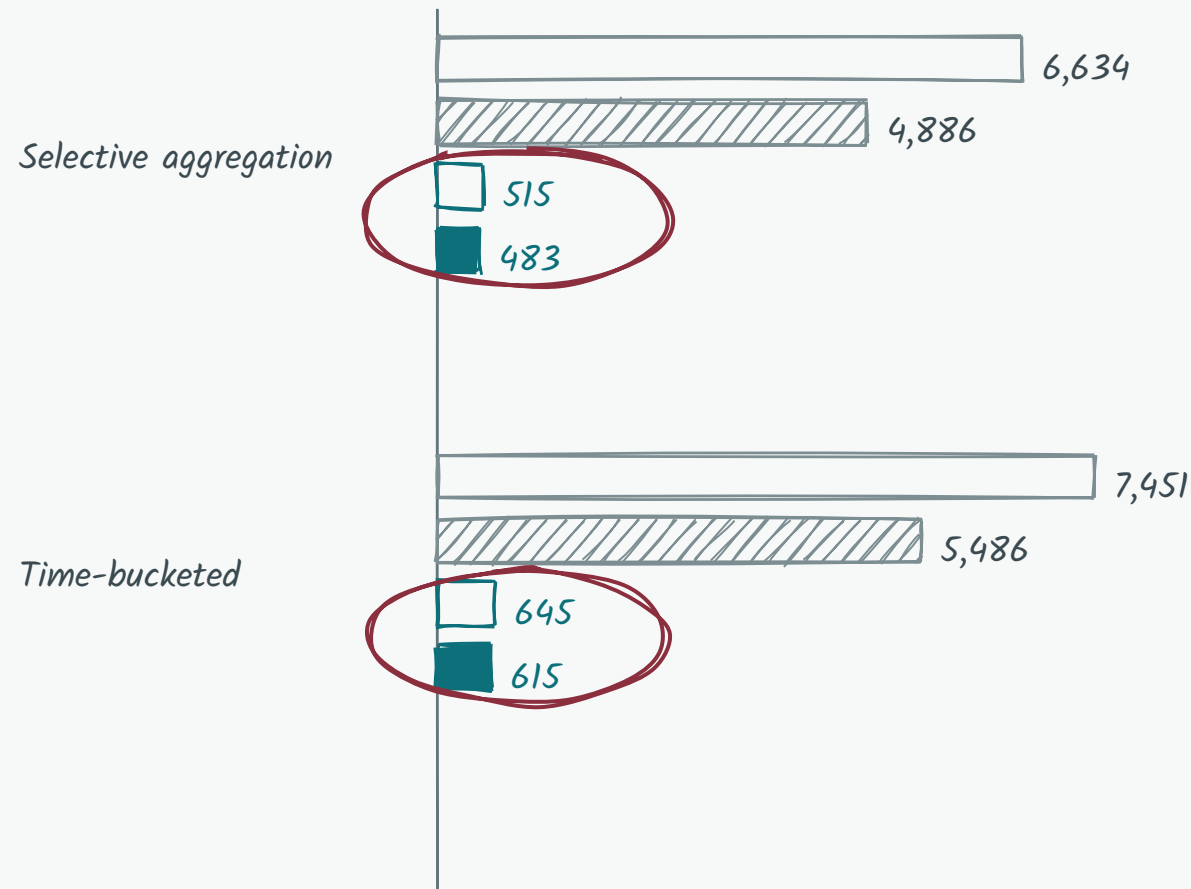
ICEBERG ♥ PINOT

STARTREE ONLY

37 / 48

Inverted indexes don't help aggregations

ClickHouse · Trino · StarTree · StarTree, pinned indexes



StarTree Indexes in action

- A different benchmark
- Similar query: filter + SUM
- Star-tree: pre-aggregated rows
- ~2,850× fewer rows read

ROWS READ PER QUERY · TPC-H SF100, 600 M ROWS

Range indexes only



inferred: 114,160 measured on SF1 × 100

+ star-tree

4,002 · ~2,850× fewer

~12 k QPS

at 148 ms P90, 12 servers



startree.ai · 125k QPS on open tables

ICEBERG ♥ PINOT

*Not the Trino and ClickHouse benchmark: TPC-H Q6
on Parquet in S3, External Tables, 12 × i4g.4xlarge.*

STARTREE ONLY

39 / 48

CHAPTER 4

Wrapping up

When it fits, and how to use it well



How engines read Iceberg, completed

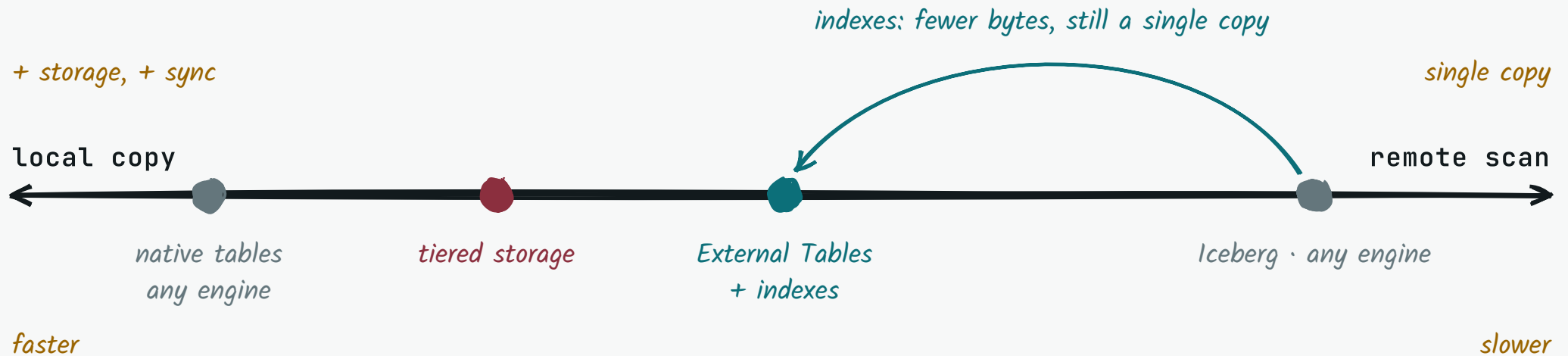
ENGINE	LOCAL COPY	OWN INDEXES ON REMOTE DATA	REMOTE SCAN + CACHE
StarTree Pinot	✓ native	✓ External Tables + indexes	✓ External Tables
Snowflake	✓	✓ search optimization*	✓
Starburst (Warp Speed)	—	✓ index layer	✓
BigQuery	✓	? external: ✗	✓
Databricks	✓	? Reyden undisclosed	✓
Trino / Athena / Spark	—	✗	✓
DuckDB / Dremio	✓	✗	✓
StarRocks / Doris / ClickHouse	✓	✗	✓



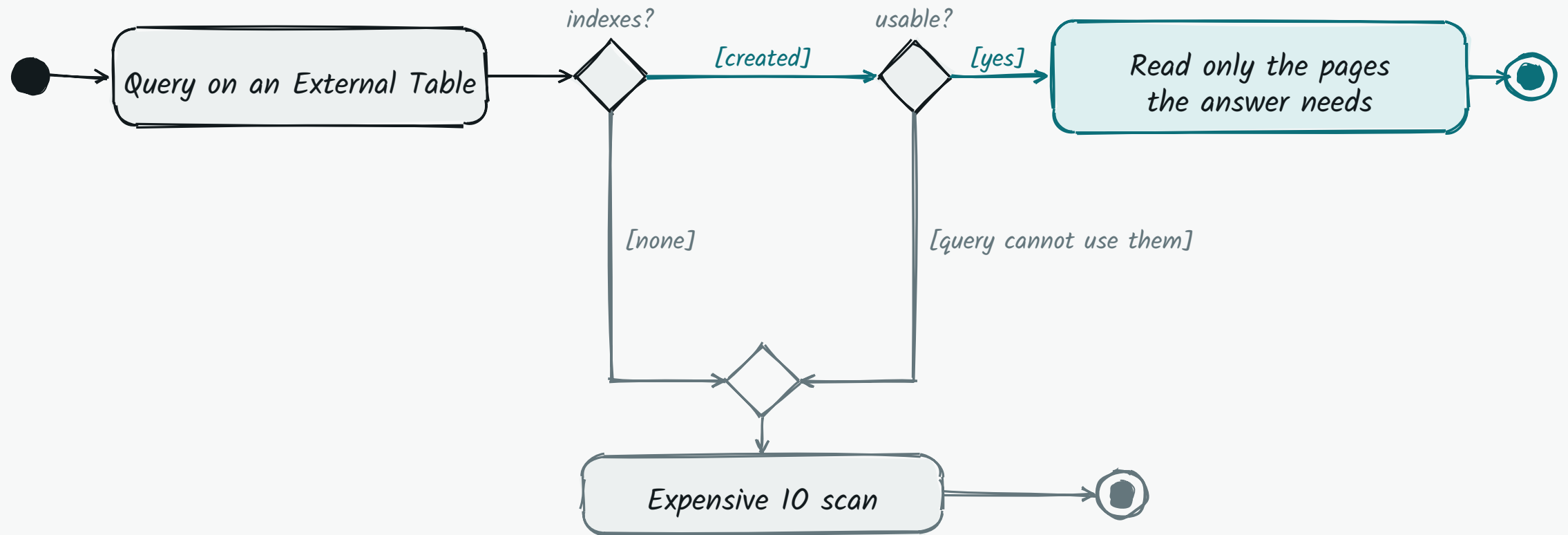
* Snowflake-managed Iceberg tables confirmed; externally managed not documented. Official docs, checked 2026-10-01.

Spotify Engineering • 2026

You cannot beat physics

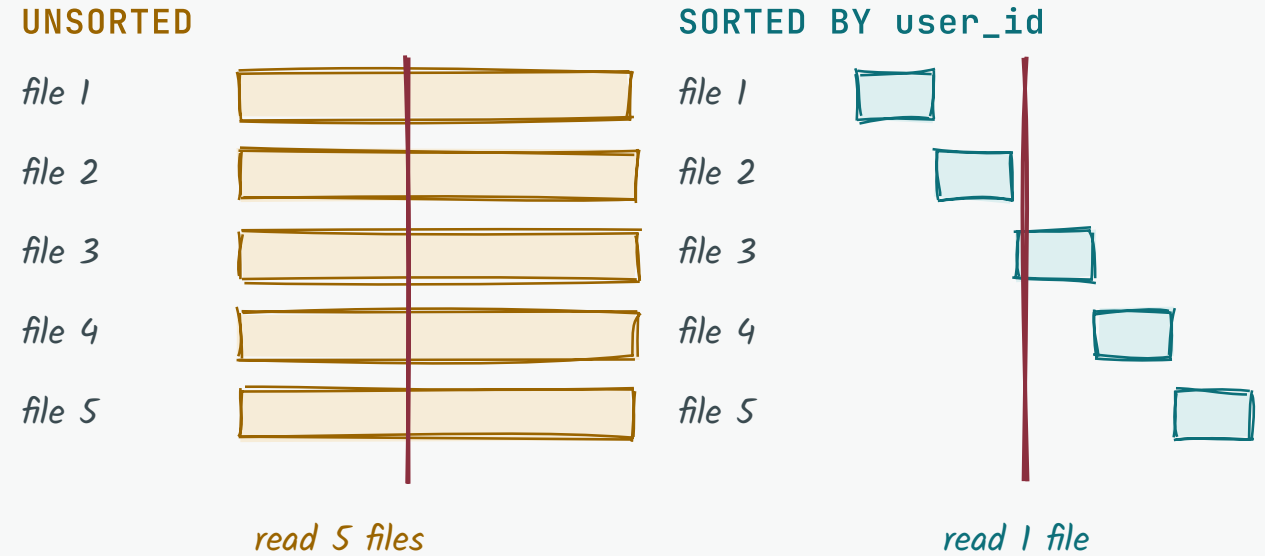


You cannot always use indexes



Write Iceberg for its readers

- Sort by the filter key
- Sane file and row-group sizes
- v3 deletion vectors
- Compact on a schedule



also: target file size · row-group size

prefer v3 deletion vectors

compact on a schedule

helps every engine, not only Pinot

Configure Pinot for your queries

- Index what you filter on
- Star-tree for dashboards
- Size the cache
- Same region as the bucket



Index what you filter on

one read per column per segment



Star-tree for dashboards

answers without rows



Size the cache

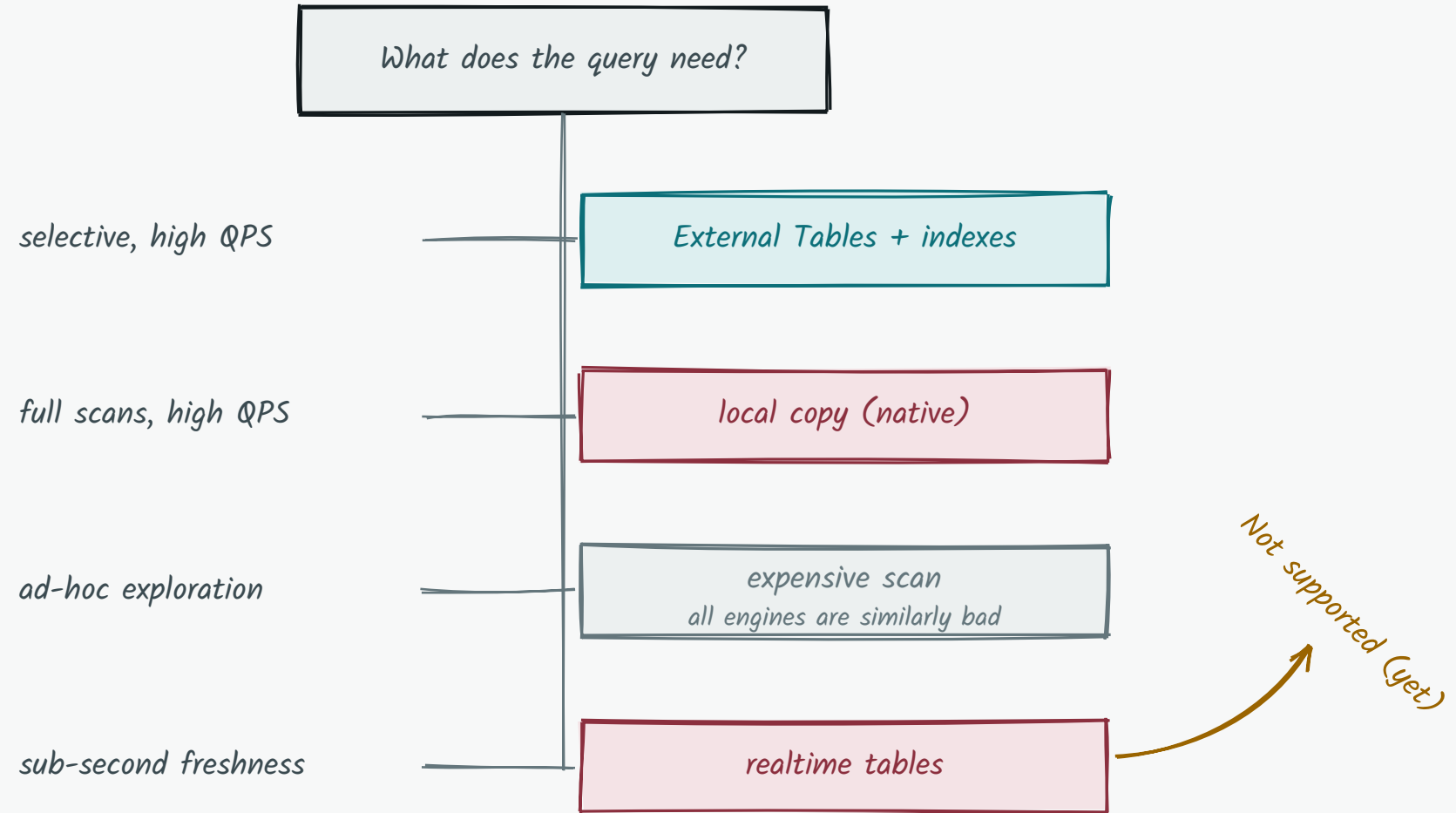
pin indexes first



Same region as the bucket

ranged GETs are region-strict

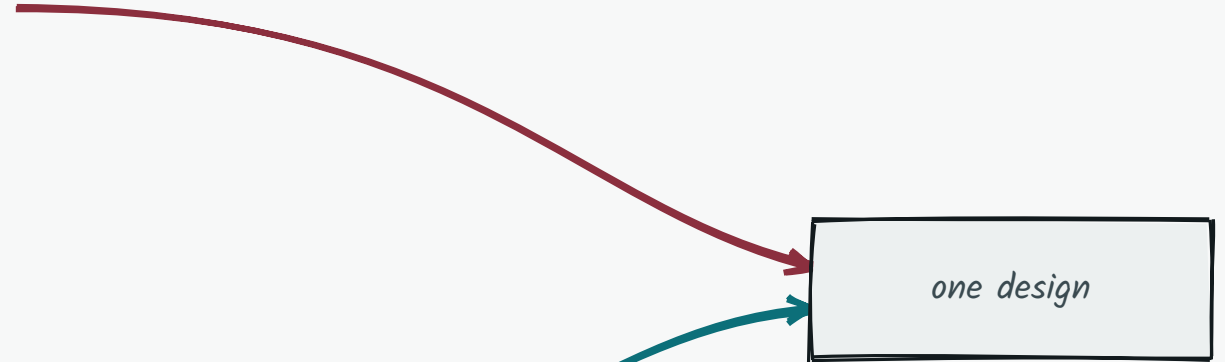
Pick the mode



Questions

- Iceberg: format, not engine
- Pinot: indexes for every user
- Together: no copy, when it prunes

Pinot · 2013



Iceberg · 2017

Links



gortizja.com/talks

These slides



youtube.com

Iceberg in 10 minutes



inviter.co

Apache Pinot Slack



pinot.apache.org

Apache Pinot



engineering.atspotify.com

Spotify: indexing the lake



startree.ai

**Benchmark vs Trino,
ClickHouse**



startree.ai

125k QPS on open tables



startree.ai

StarTree Pinot on Iceberg